Error Handling

Hostel Management System (HMS) — Software Requirements Specification

Purpose

Define consistent, safe, and understandable handling of validation failures, authorization failures, missing records, conflicts, and unexpected system faults across HMS modules.

General requirements

- Use a consistent error format across application modules and service operations.
- Distinguish user-correctable errors (such as invalid input) from access errors, missing resources, business-rule conflicts, and unexpected server failures.
- Provide a clear message and, where useful, a stable error code and field-level details. Do not expose stack traces, database queries, file paths, secrets, or internal implementation details to end users.

Expected error categories

- Validation error: one or more submitted values do not meet the field or business rules. Identify the affected fields and explain how to correct them.
- Authentication error: credentials are invalid, the account is inactive, or authentication is required. Do not disclose whether a particular account exists.
- Authorization error: the authenticated user lacks permission for the requested action or record.
- Not found: the requested student, room, bed, application, payment, complaint, visitor, or other resource does not exist or is not visible to the caller.
- Conflict: the requested operation conflicts with current state, such as allocating an already occupied bed, approving an already processed application, or checking out a student who is not currently checked in.
- Unexpected failure: an unhandled application, database, or dependency error occurred. Return a generic message and log diagnostic details securely.

Data integrity and recovery

- Do not partially apply multi-step operations. Use database transactions for related changes such as assigning a bed and updating its availability.
- On transaction failure, roll back the incomplete operation and preserve a consistent state.
- For retryable operations, use safe retry or idempotency handling where duplicate submissions could create duplicate applications, payments, or audit events.
- Do not automatically retry non-idempotent operations unless duplicate prevention is in place.

Logging and user feedback

- Log unexpected failures with a correlation/request identifier, timestamp, affected module, and diagnostic context while excluding passwords, tokens, and unnecessary personal data.
- Show users the correlation identifier when support follow-up may be needed.
- Use actionable messages and preserve form values where safe so users do not need to re-enter valid information.

Operational behavior

- Set appropriate timeouts for database and external-service calls.
- Handle unavailable dependencies gracefully and avoid presenting an unconfirmed payment or allocation as successful.
- Provide a controlled maintenance/unavailable response when the service cannot safely process requests.

Acceptance criteria

- Invalid input receives a clear validation response.
- Permission failures do not disclose protected data.
- Unexpected failures show a safe generic message and are logged for diagnosis.
- Failed multi-step operations leave no partial or inconsistent records.
- Duplicate submissions do not unintentionally create duplicate business records.

Document note: This document defines project-level requirements for the HMS utility. Confirm implementation-specific settings (such as role names, file limits, retention periods, and timeout values) against the approved project configuration before production deployment.